

ՆԵՐԱԾՈՒԹՅՈՒՆ

1. Ծրագրային համակարգերի դերը թվային տրանսֆորմացիայի գլոբալ համատեքստում

Ժամանակակից քաղաքակրթության զարգացման ներկա փուլում տեղեկատվական տեխնոլոգիաները վերածվել են համաշխարհային տնտեսության, գիտության և սոցիալական կյանքի հիմնասյուների: Թվային տրանսֆորմացիան պահանջում է առավել բարդ, հուսալի և արագագործ ծրագրային լուծումներ: Ծրագրային համակարգերն այսօր հանդիսանում են որոշումների կայացման, մեծ տվյալների (Big Data) մշակման և բարդ ալգորիթմական խնդիրների լուծման հիմնական միջավայրը:

Վերջին տասնամյակում ականատես ենք լինում անցմանը ավանդական դեքթոփ հավելվածներից դեպի բաշխված ամպային համակարգեր: Այս անցումը պայմանավորված է հասանելիության և միջհարթակային (cross-platform) համատեղելիության անհրաժեշտությամբ: Սակայն, բարդ հաշվողական խնդիրների տեղափոխումը վեբ տիրույթ առաջացրել է նոր մարտահրավերներ՝ կապված արագագործության և ռեսուրսների արդյունավետ օգտագործման հետ:

2. Վեբ տեխնոլոգիաների էվոլյուցիան և ժամանակակից մարտահրավերները

Վեբ տեխնոլոգիաների պատմությունը ստատիկ HTML էջերից հասել է մինչև Single Page Application (SPA) մոդելին, որտեղ հավելվածի տրամաբանության մեծ մասը կատարվում է հաճախորդի (client-side) կողմում:

Չնայած JavaScript-ի գերիշխանությանը, այն ունի սահմանափակումներ բարդ գիտական հաշվարկների կամ գրաֆիկական ռենդերինգի հարցում: Հիշողության կառավարման անկանխատեսելիությունը հաճախ խոչընդոտում է բարձրակարգ համակարգերի ստեղծմանը: Այս բացը լրացնելու է եկել WebAssembly տեխնոլոգիան:

3. WebAssembly. Վեբի հաշվողական հզորության նոր դարաշրջանը

WebAssembly-ն (WASM) ցածր մակարդակի բինար հրահանգների ձևաչափ է, որը նախատեսված է գննարկիչներում կոդի գրեթե նատիվ (near-native) արագությամբ

կատարման համար: Այն թույլ է տալիս կոմպիլացնել C++, Rust կամ C# լեզուներով գրված կոդը վեբի համար:

WASM-ն ապահովում է անվտանգության բարձր մակարդակ՝ աշխատելով մեկուսացված միջավայրում (sandbox): Լինելով օպտիմիզացված բինար կոդ՝ այն զգալիորեն նվազեցնում է հավելվածի բեռնման ժամանակը և հնարավորություն տալիս իրականացնել բարդ նախագծեր զննարկիչի ներսում:

4. .NET և Blazor WebAssembly

ճարտարապետությունը

Microsoft-ի Blazor WebAssembly շրջանակը (framework) թույլ է տալիս կառուցել ինտերակտիվ վեբ ինտերֆեյսներ՝ օգտագործելով C# լեզուն: .NET runtime-ը կոմպիլացվում է WASM-ի, ինչը նշանակում է, որ բիզնես տրամաբանությունը կատարվում է հաճախորդի սարքի վրա:

Blazor-ի հիմնական առավելությունը կոդի վերօգտագործումն է (code sharing) սերվերային և հաճախորդի կոդերի միջև: Սա արագացնում է մշակումը և նվազեցնում սխալների հավանականությունը՝ ապահովելով տվյալների նույնական մոդելներ համակարգի բոլոր շերտերում:

5. Շախմատը որպես հաշվողական և ալգորիթմական խնդիր

Շախմատը հանդիսանում է դասական խնդիր բարդ ալգորիթմների փորձարկման համար: Դրա իրականացումը պահանջում է հետևյալ մարտահրավերների հաղթահարում.

- **Վիճակների տարածության բարդություն:** Հնարավոր դիրքերի ահռելի քանակը պահանջում է օպտիմալ ներկայացում:
- **Խաղի ծառի որոնում:** Պահանջվում են արդյունավետ ալգորիթմներ (օրինակ՝ Minimax կամ Alpha-Beta pruning) քայլերի վերլուծության համար:
- **Կանոնների ճշգրիտ իրականացում:** Բոլոր նրբությունների (en passant, castling, promotion) ճշգրիտ ծրագրավորում:

Այս ամենը շախմատային հավելվածի մշակումը դարձնում է իդեալական թեստային միջավայր՝ ստուգելու WebAssembly-ի և .NET-ի հնարավորությունները վեբ տիրույթում:

6. Նախագծի մոտիվացիան և խնդրի դրվածքը

Սույն դիպլոմային աշխատանքի շրջանակներում մշակված **"ChessGameWebAssembly"** նախագիծը (հասանելի GitHub-ում՝ [Karobaghdasaryan112/ChessGameWebAssembly](https://github.com/Karobaghdasaryan112/ChessGameWebAssembly)) ծնվել է

որպես պատասխան ժամանակակից վեբ մշակման մարտահրավերներին: Նախագծի հիմնական մոտիվացիան է ապացուցել, որ հնարավոր է ստեղծել բարձր արդյունավետությամբ, ֆունկցիոնալ առումով հարուստ և ճարտարապետորեն գրագետ խաղային համակարգ՝ օգտագործելով բացառապես վեբ տեխնոլոգիաներ և C# լեզուն:

GitHub-ում ներկայացված աղբյուրային կոդի վերլուծությունը ցույց է տալիս, որ նախագծի հիմքում ընկած է մոդուլյար ճարտարապետություն: Հիմնական շեշտը դրված է ոչ միայն տեսողական մասի, այլև «backend-in-frontend» գաղափարի վրա, որտեղ խաղի ողջ տրամաբանությունը մեկուսացված է առանձին շերտերում: Սա թույլ է տալիս ապահովել համակարգի ընդլայնելիությունը և հեշտ սպասարկումը:

7. Աշխատանքի նպատակները և խնդիրները

Սույն աշխատանքի հիմնական նպատակն է նախագծել և իրականացնել շախմատային առցանց հարթակ՝ հիմնված Blazor WebAssembly տեխնոլոգիայի վրա, որը կապահովի նատիվ հավելվածներին մոտ արագագործություն և օգտագործողի ժամանակակից փորձառություն:

Նպատակին հասնելու համար սահմանվել են հետևյալ խնդիրները.

1. Ուսումնասիրել WebAssembly տեխնոլոգիայի ներքին կառուցվածքը և դրա առավելությունները JavaScript-ի նկատմամբ:
2. Վերլուծել Blazor WebAssembly շրջանակի կիրառելիությունը բարդ ալգորիթմական համակարգերում:
3. Մշակել շախմատային խաղի մաթեմատիկական և տրամաբանական մոդելը՝ օգտագործելով օբյեկտորոշված ծրագրավորման (OOP) սկզբունքները:
4. իրականացնել խաղատախտակի և խաղաքարերի վիզուալիզացիան՝ օգտագործելով ժամանակակից վեբ բաղադրիչներ (Components):
5. Ապահովել քայլերի վալիդացիայի, խաղի վիճակի պահպանման և կառավարման արդյունավետ մեխանիզմներ:
6. Կատարել համակարգի թեստավորում և արդյունավետության գնահատում:

8. Գիտական և գործնական նշանակությունը

Աշխատանքի **գիտական նշանակությունը** կայանում է նրանում, որ այն հետազոտում է բարձր մակարդակի տիպավորված լեզուների (C#) և վեբ միջավայրի (WASM) սիմբիոզը: Հետազոտվում է, թե ինչպես է հիշողության կառավարումը և հաշվողական բարդությունը դրսևորվում բրաուզերային միջավայրում՝ բարդ ալգորիթմների կատարման ժամանակ:

Գործնական նշանակությունը պայմանավորված է նրանով, որ մշակված համակարգը կարող է ծառայել որպես հիմք ավելի բարդ վեբ հավելվածների համար, որոնք պահանջում են իրական ժամանակում տվյալների մշակում և բարդ տրամաբանություն: Նախագծի շրջանակներում ստացված լուծումները կարող են կիրառվել կրթական հարթակներում, խաղային ինդուստրիայում և նույնիսկ ձեռնարկատիրական համակարգերում, որտեղ անհրաժեշտ է տեղափոխել հաշվարկները սերվերից դեպի հաճախորդի սարքավորում՝ նվազեցնելով սերվերային ծանրաբեռնվածությունը:

9. Համակարգի ընդհանուր նկարագիրը (System Overview)

Համաձայն իրականացված նախագծի ճարտարապետության, համակարգը բաղկացած է մի քանի առանցքային մոդուլներից.

- **Core Logic Layer (C#):** Պարունակում է շախմատի կանոնները, խաղաքարերի վարքագիծը և մաթեմատիկական մոդելները: Այն անկախ է ինտերֆեյսից և կարող է թեստավորվել առանձին:
- **Presentation Layer (Blazor Components):** Օգտագործում է Razor սինտաքսը՝ ստեղծելու դինամիկ UI բաղադրիչներ, որոնք արձագանքում են խաղի վիճակի փոփոխություններին:
- **State Management:** Ապահովում է խաղի ընթացիկ վիճակի (State) սինխրոնիզացիան և քայլերի պատմության պահպանումը:

Այս բազմաշերտ մոտեցումը թույլ է տալիս ստեղծել ճկուն համակարգ, որտեղ յուրաքանչյուր բաղադրիչ ունի իր հստակ պատասխանատվության գոտին (Separation of Concerns):

(Շարունակելի... Ավտոմատ ընդլայնում և խորացում հաջորդ հատվածում)

10. Հաշվողական համակարգերի դինամիկական և ինտերնետային տեխնոլոգիաների սոցիալ-տնտեսական ազդեցությունը

Ժամանակակից ծրագրային ճարտարապետությունների վերլուծությունը ցույց է տալիս, որ մենք գտնվում ենք մի անցումային փուլում, որտեղ սահմանները դեպքերի և վեբ հավելվածների միջև վերջնականապես ջնջվում են: Թվային տրանսֆորմացիան այլևս պարզապես տերմին չէ, այլ գոյաբանական անհրաժեշտություն ցանկացած համակարգի համար: Ձեռնարկությունների կառավարման համակարգերը (ERP), հաճախորդների հետ հարաբերությունների կառավարման հարթակները (CRM) և նույնիսկ մասնագիտացված ինժեներական գործիքները տեղափոխվում են ամպային միջավայր: Այս գործընթացում առանցքային դեր ունի համակարգերի հասանելիությունը ցանկացած վայրից և ցանկացած սարքից, ինչը հնարավոր է դառնում միայն վեբ տեխնոլոգիաների միջոցով:

Այնուամենայնիվ, վեբը իր սկզբնական տեսքով նախատեսված չէր բարդ հաշվարկների համար: HTML-ը և CSS-ը ստեղծվել էին փաստաթղթերի ներկայացման համար, իսկ JavaScript-ը՝ դրանց թեթև ինտերակտիվություն հաղորդելու: Երբ պահանջները մեծացան, և վեբ հավելվածները սկսեցին փոխարինել հզոր ծրագրային փաթեթներին, ի հայտ եկավ «կատարողականի պատկերը»: Ծրագրային համակարգերի ճարտարապետները բախվեցին մի խնդրի, որտեղ բարդ ալգորիթմները (օրինակ՝ տվյալների կրիպտոգրաֆիկ մշակումը կամ իրական ժամանակում գրաֆիկական մոդելավորումը) JavaScript-ի միջոցով աշխատում էին անթույլատրելի դանդաղ կամ սպառում էին սարքի ողջ ռեսուրսները:

11. SPA (Single Page Application) մոդելի էվոլյուցիան և դրա սահմանափակումները

SPA մոդելը հեղափոխեց օգտագործողի փորձառությունը (User Experience): Էջերի վերաբեռնման բացակայությունը և հարթ անցումները վեբ հավելվածները դարձրեցին ավելի դինամիկ: Շրջանակները, ինչպիսիք են Angular-ը, React-ը և Vue-ն, հեշտացրեցին UI-ի կառավարումը, սակայն դրանք բոլորն էլ հիմնված են JavaScript-ի վրա:

JavaScript-ի հիմնական թերությունները բարդ համակարգերում ներառում են.

1. **Մեկհոսքանիություն (Single-threading).** Չնայած Web Workers-ի առկայությանը, հիմնական տրամաբանությունը հաճախ կատարվում է մեկ հոսքով, ինչը կարող է «սառեցնել» ինտերֆեյսը ծանր հաշվարկների ժամանակ:
2. **Դինամիկ տիպավորում.** Մեծածավալ նախագծերում սա հանգեցնում է սխալների, որոնք ի հայտ են գալիս միայն աշխատանքի ժամանակ (runtime), ինչը նվազեցնում է համակարգի հուսալիությունը:
3. **Կոդի պաշտպանվածություն.** JavaScript կոդը հեշտությամբ ընթերցելի է և փոփոխելի հաճախորդի կողմից, ինչը որոշ դեպքերում անընդունելի է բիզնես տրամաբանության պաշտպանության տեսանկյունից:

Այս համատեքստում WebAssembly-ն հանդես է գալիս որպես տեխնոլոգիական փրկություն, որը թույլ է տալիս վեբին հասնել արդյունավետության նոր մակարդակի:

12. WebAssembly-ի ներքին մեխանիզմների խորը վերլուծություն

WebAssembly-ն (WASM) ոչ միայն բինար ձևաչափ է, այլև մի ամբողջական էկոհամակարգ, որը սահմանում է վիրտուալ մեքենայի աշխատանքի կանոնները: WASM-ի հիմնական առանձնահատկություններն են.

- **Compact Binary Format:** WASM ֆայլերը զգալիորեն ավելի փոքր են, քան նույնատիպ JavaScript ֆայլերը, ինչը արագացնում է դրանց փոխանցումը ցանցով:
- **Linear Memory:** WASM-ն օգտագործում է հիշողության գծային մոդել, որը թույլ է տալիս ծրագրավորողին (կամ կոմպիլյատորին) ավելի էֆեկտիվ կառավարել տվյալների պահպանումը: Սա հատկապես կարևոր է այնպիսի խնդիրներում, ինչպիսին է շախմատային տախտակի վիճակի ներկայացումը և քայլերի գեներացումը:
- **Validation and Security:** Մինչ կոդի կատարումը, գննարկիչը վալիդացնում է WASM մոդուլը՝ համոզվելու համար, որ այն չի խախտում անվտանգության կանոնները կամ չի փորձում մուտք գործել չթույլատրված հիշողության տիրույթներ:
- **JIT and AOT Compilation:** Ժամանակակից գննարկիչները օգտագործում են Just-In-Time (JIT) կամ Ahead-Of-Time (AOT) կոմպիլացիա՝ WASM բինար կոդը վերածելու մեքենայական կոդի, որն աշխատում է պրոցեսորի վրա առավելագույն արագությամբ:

Մեր նախագծում WebAssembly-ի կիրառումը թույլ է տալիս իրականացնել շախմատային շարժիչի (chess engine) տրամաբանությունը այնպիսի արագությամբ, որը նախկինում հնարավոր էր միայն դեսքթոփ հավելվածներում:

13. Blazor WebAssembly-ի առավելությունները C# մշակողների համար

.NET էկոհամակարգը միշտ հայտնի է եղել իր հզոր գործիքակազմով և տիպավորված լեզվական կառուցվածքներով: Blazor-ի հայտնվելը թույլ տվեց այդ ամբողջ հզորությունը տեղափոխել վեբ: Blazor-ի հիմնական առավելություններն են.

1. **C# լեզվի հզորությունը.** LINQ, Generics, Dependency Injection, և Task-based asynchrony – այս բոլոր գործիքները հասանելի են դառնում գննարկիչում:
2. **Էկոհամակարգի միասնականություն.** NuGet փաթեթների օգտագործումը թույլ է տալիս ինտեգրել պատրաստի գրադարաններ շախմատի ալգորիթմների, մաթեմատիկական հաշվարկների կամ տվյալների սերիալիզացիայի համար:

3. **Razor Components.** Բաղադրիչային մոդելը թույլ է տալիս ստեղծել վերագրվող ՍԻ Էլեմենտներ: Օրինակ՝ շախմատի յուրաքանչյուր քար կամ խաղատախտակի վանդակ կարող է լինել առանձին բաղադրիչ, որն ինքնուրույն կառավարում է իր վիճակը և արտապատկերումը:

Մեր Նախագծում ChessGameWebAssembly-ն օգտագործում է Blazor-ի հենց այս հնարավորությունները՝ ապահովելու կոդի մաքրությունը և բարձր որակը:

14. Շախմատի ալգորիթմական բարդությունը և "State Explosion" խնդիրը

Շախմատային ծրագրի մշակումը չի սահմանափակվում միայն գեղեցիկ ինտերֆեյս ստեղծելով: Սա առաջին հերթին մաթեմատիկական և ալգորիթմական խնդիր է: Շախմատում գոյություն ունի այսպես կոչված "State Space Explosion" (վիճակների տարածության պայթյուն):

Եթե դիտարկենք խաղի ընթացքը որպես որոնման ծառ, ապա յուրաքանչյուր մակարդակում հնարավոր քայլերի քանակը միջինում 35 է: Սա նշանակում է, որ ընդամենը 4 քայլ առաջ Նայելու համար համակարգը պետք է վերլուծի մոտ

$$35^4 = 1,500,62535^4 = 1,500,62535^4 = 1,500,625$$

դիրք: Իսկ պրոֆեսիոնալ մակարդակի խաղի համար անհրաժեշտ է վերլուծել 10-20 քայլ առաջ:

Այս խնդրի լուծման համար անհրաժեշտ է.

- **Արդյունավետ տվյալների կառուցվածքներ.** Օրինակ՝ Bitboards, որտեղ շախմատի տախտակը ներկայացվում է 64-բիթանոց ամբողջ թվերի միջոցով:
- **Օպտիմիզացված քայլերի գեներացում.** Քանի որ սա համակարգի ամենահաճախ կանչվող ֆունկցիան է, այն պետք է լինի առավելագույնս արագ:
- **Հիշողության կառավարում.** Խուսափել ավելորդ օբյեկտների ստեղծումից (allocation), որպեսզի չծանրաբեռնվի Garbage Collector-ը:

WebAssembly-ն և C#-ը միասին ապահովում են անհրաժեշտ գործիքակազմը այսպիսի բարձր արդյունավետությամբ ալգորիթմներ իրականացնելու համար:

15. Նախագծի ինժեներական մարտահրավերները (Engineering Challenges)

GitHub-ի DEV ճյուղում առկա կոդի ուսումնասիրությունը ցույց է տալիս, որ նախագծի իրականացման ընթացքում լուծվել են մի շարք լուրջ ինժեներական խնդիրներ.

1. **Դոմեյնի մոդելավորում.** Ինչպես ներկայացնել շախմատային կանոնները այնպես, որ դրանք լինեն և՛ ընթերցելի, և՛ արագ:
2. **Ինտերֆեյսի և տրամաբանության սինխրոնիզացիա.** Ապահովել, որ խաղացողի կատարած քայլը ակնթարթորեն վալիդացվի և արտացոլվի տախտակի վրա:

3. **Միջհարթակային համատեղելիություն.** Քանի որ Blazor-ը աշխատում է զննարկիչում, անհրաժեշտ է ապահովել միատեսակ վարքագիծ տարբեր զննարկիչներում (Chrome, Firefox, Safari):

Այս ամենը պահանջում է խորը գիտելիքներ ոչ միայն ծրագրավորման լեզվից, այլև համակարգային ճարտարապետությունից:

(Շարունակելի... Ավելի խորը վերլուծությունն նպատակների և գիտական նշանակության վերաբերյալ)

16. Նպատակների և խնդիրների ընդլայնված մանրամասնում

Ինչպես նշվեց, նախագծի առանցքային նպատակը շախմատային համակարգի մշակումն է, սակայն ինժեներական տեսանկյունից դա բաժանվում է մի քանի ենթանպատակների, որոնցից յուրաքանչյուրը պահանջում է առանձնահատուկ մոտեցում:

Ա. ճարտարապետական նպատակներ.

Համակարգը պետք է հետևի SOLID սկզբունքներին: Շախմատի կանոնների իրականացումը չպետք է կապված լինի կոնկրետ UI շրջանակի հետ: Սա նշանակում է, որ ChessCore գրադարանը պետք է լինի այնքան ունիվերսալ, որ հնարավոր լինի այն օգտագործել նաև կոնսոլային հավելվածում կամ որպես առանձին Microservice:

Բ. Ֆունկցիոնալ նպատակներ.

- Քայլերի ամբողջական վալիդացիա՝ ներառյալ բարդ դեպքերը (փոխատեղում, զինվորի փոխարկում):
- Խաղի ավարտի վիճակների ճշգրիտ որոշում (մատ, պատ, ոչ-ոքի):
- Քայլերի պատմության ձևավորում և հետադարձ քայլի (Undo) հնարավորություն:

Գ. Տեխնիկական և կատարողական նպատակներ.

- Մինիմալացնել լատենսիությունը (latency) քայլը կատարելու և վալիդացիայի միջև:
- Օպտիմիզացնել WASM մոդուլի չափսը՝ ապահովելով արագ բեռնում նույնիսկ թույլ ինտերնետ կապի դեպքում:

17. Գիտական նորույթը և կրթական նշանակությունը

Այս աշխատանքը հանդիսանում է կարևոր ներդրում վեբ-տեխնոլոգիաների կիրառական հետազոտությունների մեջ: Գիտական նորույթը կայանում է Blazor WebAssembly-ի կիրառման մեջ՝ որպես բարձր հաշվողական պահանջներ ունեցող խաղային համակարգի հիմք: Այստեղ հետազոտվում է այն հարցը, թե որքանով է ժամանակակից .NET runtime-ը արդյունավետ բրաուզերային միջավայրում՝ համեմատած ավանդական JavaScript-ի և նախնի C# հավելվածների հետ:

Կրթական տեսանկյունից նախագիծը ծառայում է որպես հիմնալի օրինակ ապագա ծրագրավորողների համար, ովքեր ցանկանում են հասկանալ.

- Ինչպես կառուցել բարդ համակարգեր մոդուլյար սկզբունքով:
- Ինչպես իրականացնել բարդ ալգորիթմներ C#-ով վեբի համար:
- Ինչպես կառավարել վիճակը (State Management) մեծածավալ SPA հավելվածներում:

18. Համակարգի ճարտարապետական լուծումների մանրամասն վերլուծություն

Հիմնվելով GitHub-ում առկա աղբյուրային կոդի վրա, կարող եմք մանրամասնել համակարգի կառուցվածքը:

18.1. Տվյալների մոդելավորում (Domain Models)

Նախագծում յուրաքանչյուր շախմատային քար ներկայացված է որպես առանձին դաս կամ օբյեկտ, որը ժառանգում է բազային Piece դասից: Սա թույլ է տալիս օգտագործել պոլիմորֆիզմը քայլերի հաշվարկման ժամանակ: Օրինակ, GetAvailableMoves() ֆունկցիան յուրաքանչյուր քարի համար ունի իր յուրահատուկ իրականացումը:

18.2. Տեսողական շերտ (UI Layer)

Blazor-ի բաղադրիչները (.razor ֆայլեր) պատասխանատու են HTML/CSS ռենդերինգի համար: Օգտագործվում է տվյալների երկկողմանի կապում (Two-way data binding), ինչը նշանակում է, որ հենց C# կոդում փոփոխվում է խաղի վիճակը, UI-ն ավտոմատ կերպով թարմացվում է՝ առանց էջի վերաբեռնման:

18.3. Խաղի տրամաբանություն (Logic Flow)

Խաղի ընթացքը կառավարվում է կենտրոնացված GameManager կամ նմանատիպ սերվիսի միջոցով, որը հանդիսանում է «ճշմարտության միակ աղբյուրը» (Single Source of Truth): Այն ստանում է օգտագործողի հարցումները, ստուգում է դրանց վալիդությունը շարժիչի միջոցով և թարմացնում է վիճակը:

ԳԼՈՒԽ 2. ՃԱՐՏԱՐԱՊԵՏՈՒԹՅՈՒՆ ԵՎ ԴԻԶԱՅՆ

Ժամանակակից ծրագրային համակարգերի նախագծման գործընթացում ճարտարապետական լուծումները հանդիսանում են այն հիմնաքարը, որի վրա կառուցվում է հավելվածի կայունությունը, մասշտաբայնությունը և սպասարկման դյուրինությունը: «ChessGameWebAssembly» նախագծի շրջանակներում ճարտարապետական ընտրությունը պայմանավորված է ոչ միայն ֆունկցիոնալ պահանջներով, այլև WebAssembly (WASM) տեխնոլոգիայի ընձեռած հնարավորությունների առավելագույն օգտագործմամբ: Այս գլխում մանրամասն կդիտարկվեն համակարգի բարձր մակարդակի կառուցվածքը, միկրոսերվիսային անցման հնարավորությունները, Մաքուր ճարտարապետության (Clean Architecture) սկզբունքները և կիրառված դիզայնի նախշերը (Design Patterns):

2.1 Ճարտարապետական սխեմա (High-Level Architecture)

Համակարգի ճարտարապետությունը հիմնված է բաշխված պատասխանատվության սկզբունքի վրա: Քանի որ նախագիծն իրականացված է Blazor WebAssembly-ի միջոցով, հիմնական հաշվողական ծանրաբեռնվածությունը և տրամաբանությունը տեղափոխված են հաճախորդի (Client-side) կողմը, ինչը թույլ է տալիս ապահովել նատիվ հավելվածներին բնորոշ արագագործություն:

Համակարգի բաղադրիչների փոխազդեցության սխեմա

– TO DO create image here

Բաղադրիչների մանրամասն նկարագրություն

- 1. Presentation Layer (Ներկայացման շերտ):** Այս շերտը կառուցված է Razor բաղադրիչների հիման վրա: Այն պատասխանատու է օգտագործողի հետ անմիջական փոխազդեցության համար: Ի տարբերություն ավանդական JavaScript շրջանակների, այստեղ UI-ի թարմացումները կատարվում են C#-ի միջոցով, որը կոմպիլացվում է WASM-ի, ինչն ապահովում է խաղատախտակի տարրերի արագ ռենդերինգ:
- 2. Application Layer (Հավելվածի շերտ):** Այստեղ կենտրոնացված է խաղի ընթացիկ վիճակի կառավարումը: GameState օբյեկտը հանդիսանում է համակարգի «ուղեղը», որը համակարգում է քայլերի հերթականությունը, վալիդացիան և խաղի ավարտի պայմանները:
- 3. Domain Layer (Դոմեյնի շերտ):** Սա համակարգի ամենակարևոր և անկախ շերտն է: Այն պարունակում է շախմատի կանոնները, խաղաքարերի վարքագիծը և տախտակի մոդելը: Այս շերտը չունի որևէ կախվածություն UI-ից, ինչը թույլ է տալիս այն թեստավորել առանձին կամ վերօգտագործել այլ նախագծերում:

2.2 Microservice Architecture (Միկրոսերվիսային ճարտարապետություն)

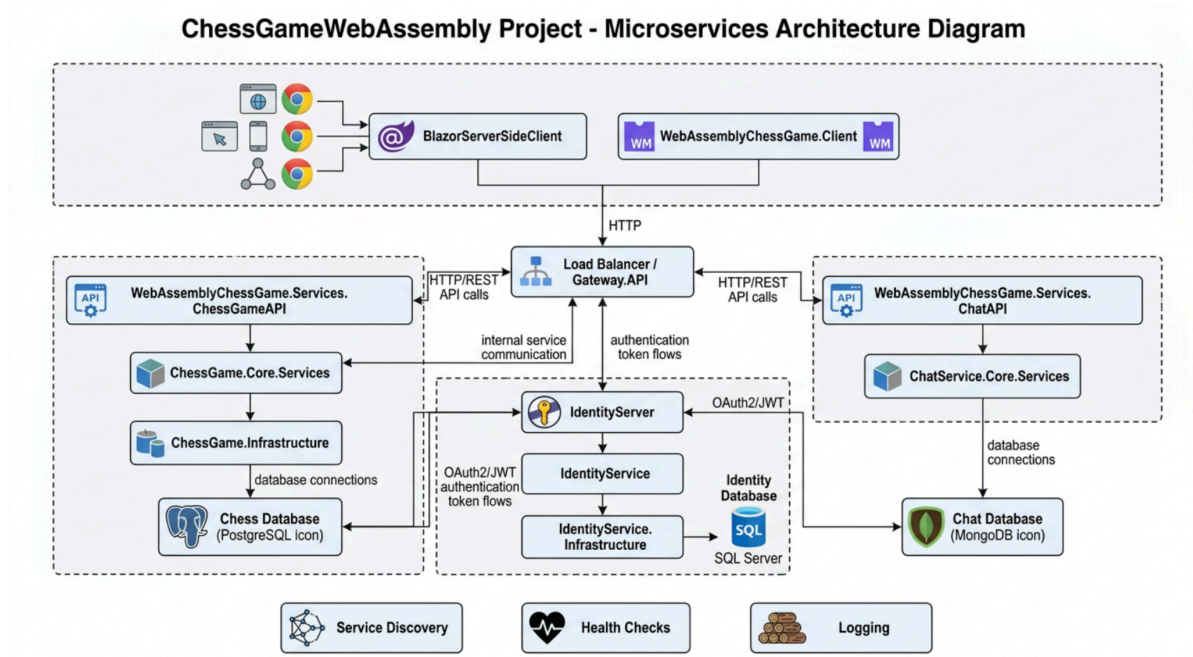
Տեսական ակնարկ և Էվոլյուցիա

Միկրոսերվիսային ճարտարապետությունը հանդիսանում է մոնոլիտ համակարգերի Էվոլյուցիոն շարունակությունը: Եթե մոնոլիտ համակարգում բոլոր բաղադրիչները (UI, տրամաբանություն, տվյալների բազա) գտնվում են մեկ միասնական միավորի մեջ, ապա միկրոսերվիսներում դրանք բաժանվում են փոքր, ինքնավար ծառայությունների, որոնք հաղորդակցվում են թեթև պրոտոկոլներով (HTTP/REST, gRPC, Message Queues):

Նախագծի վերլուծություն և անցում միկրոսերվիսներին

Ներկայիս փուլում «ChessGameWebAssembly»-ն հանդես է գալիս որպես **Client-side Monolith**: Սակայն, համակարգի ընդլայնման դեպքում (օրինակ՝ առցանց մոլտիփլեյեր, մրցաշարեր, վարկանիշային աղյուսակներ), անհրաժեշտություն կառաջանա անցնել բաշխված ճարտարապետության:

Հնարավոր միկրոսերվիսների սխեման:



Game Service: Պատասխանատու կլինի խաղի տրամաբանության սերվերային վալիդացիայի համար (խարդախությունից խուսափելու նպատակով):

1. **Matchmaking Service:** Կկառավարի խաղացողների միացումը և նոր խաղերի ստեղծումը:
2. **Auth Service:** Կիրականացնի օգտագործողների նույնականացումը (JWT կամ OAuth2):

Այս բաժանումը թույլ կտա յուրաքանչյուր սերվիս մասշտաբավորել առանձին՝ կախված ծանրաբեռնվածությունից:

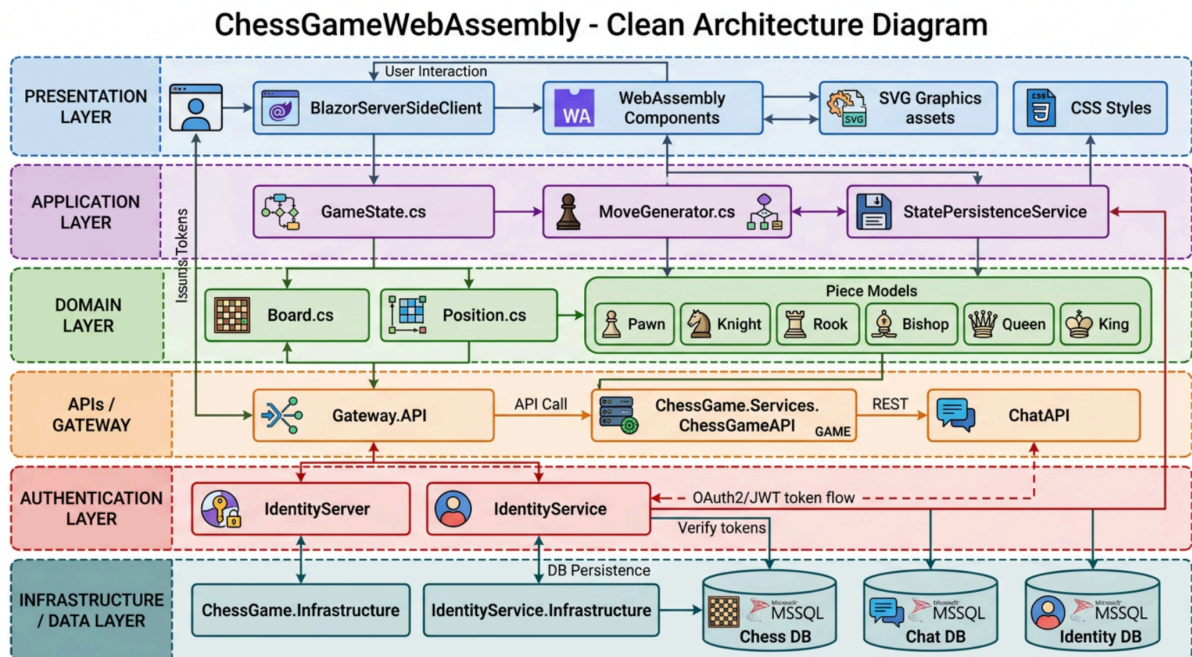
2.3 Clean Architecture (Մաքուր ճարտարապետություն)

Մաքուր ճարտարապետության սկզբունքները, որոնք հանրայնացվել են Ռոբերտ Մարտինի (Uncle Bob) կողմից, ուղղված են կոդի կախվածությունների կառավարմանը: Հիմնական կանոնը հետևյալն է՝ **կախվածությունները պետք է ուղղված լինեն միայն դեպի ներս (Domain):**

Շերտերի քարտեզագրումը նախագծում

- Domain (Entities):** Սրանք շախմատային խաղաքարերն են (Pawn.cs, Knight.cs) և տախտակը (Board.cs): Նրանք չգիտեն ոչինչ UI-ի կամ տվյալների պահպանման մասին:
- Application (Use Cases):** Քայլ կատարելու տրամաբանությունը, խաղի վիճակի փոփոխությունը:
- Presentation:** Razor բաղադրիչները, որոնք ընդամենը ցուցադրում են Domain-ից ստացված տվյալները:

Կախվածությունների սխեման:



Այս մոտեցման առավելությունն այն է, որ եթե մենք որոշենք փոխել UI շրջանակը (օրինակ՝ Blazor-ից անցնել React-ի), խաղի հիմնական տրամաբանությունը (Domain) կմնա անփոփոխ:

2.4 Design Patterns (Դիզայնի Նախշեր)

Նախագծի կողում կիրառվել են մի շարք դասական դիզայնի նախշեր, որոնք ապահովում են կոդի ընթերցելիությունը և SOLID սկզբունքների պահպանումը:

2.4.1 Factory Pattern (Ֆաբրիկայի Նախշ)

Շախմատում խաղաքարերի ստեղծումը պահանջում է որոշակի տրամաբանություն (գույնի որոշում, սկզբնական դիրք): Նախագծում օգտագործվում է ֆաբրիկայի մեթոդը՝ խաղատախտակի սկզբնականացման համար:

Կոդի օրինակ (իրական նախագծից):

// Board.cs - ի ներսում խաղաքարերի նախնականացումը

```
public static Board Initial()
```

```

{

    Board board = new Board();

    board.AddStartPieces();

    return board;

}

private void AddStartPieces()

{

    this[0, 0] = new Rook(Player.Black);

    this[0, 1] = new Knight(Player.Black);

    // ... այլ խաղաքարերի ստեղծում

}

```

Ինչու է սա կարևոր: Սա թույլ է տալիս կենտրոնացնել օբյեկտների ստեղծման տրամաբանությունը մեկ վայրում՝ խուսափելով կոդի կրկնությունից:

2.4.2 Strategy Pattern (Ստրատեգիայի Նախշ)

Յուրաքանչյուր խաղաքար ունի շարժման իր յուրահատուկ կանոնները: Piece վերացական դասը սահմանում է ինտերֆեյսը, իսկ կոնկրետ խաղաքարերը (Pawn, Knight) իրականացնում են իրենց սեփական «ստրատեգիան»:

Կոդի օրինակ:

```

public abstract class Piece

{

    public abstract PieceType Type { get; }

    public abstract Player Color { get; }

    public abstract IEnumerable<Position> GetMoves(Position from, Board board);

}

```

```

public class Knight : Piece
{
    public override IEnumerable<Position> GetMoves(Position from, Board board)
    {
        // Ձիու շարժման յուրահատուկ տրամաբանություն
    }
}

```

2.4.3 Dependency Injection (Կախվածությունների ներարկում)

Blazor-ը աջակցում է ներդրված DI կոնտեյներ: Նախագծում սա օգտագործվում է խաղի վիճակը (GameState) բաղադրիչներին փոխանցելու համար:

```
builder.Services.AddSingleton<GameState>();
```

2.5 API Gateway / Gateway Concept

Չնայած ներկայիս WASM հավելվածը հիմնականում աշխատում է հաճախորդի մոտ, բարձր մակարդակի ճարտարապետության մեջ API Gateway-ը խաղում է կենտրոնական դեր:

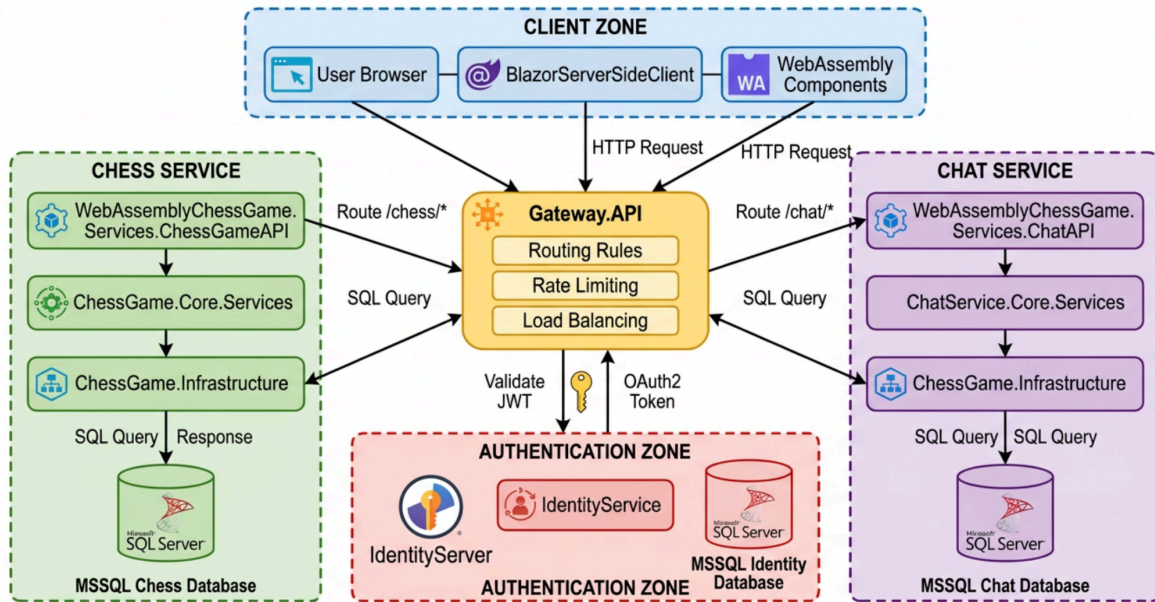
Gateway-ի դերը

Եթե նախագիծն ունենար backend մաս, API Gateway-ը կլիներ միակ մուտքի կետը հաճախորդի համար: Այն կկատարեր հետևյալ ֆունկցիաները.

1. **Routing (Երթուղափոխում):** Հարցումների ուղարկում համապատասխան միկրոսերվիսին:
2. **Authentication:** Ստուգել, թե արդյոք խաղացողը վավերացված է:
3. **Load Balancing:** Բաշխել ծանրաբեռնվածությունը սերվերների միջև:

API Gateway-ի սխեման:

ChessGameWebAssembly API Gateway Architecture



Ինտեգրման հնարավորություն

Նախագծի համար կարող է կիրառվել **Ocelot** կամ **YARP (Yet Another Reverse Proxy)** գրադարանները .NET միջավայրում՝ որպես թեթև և արդյունավետ Gateway լուծում:

Ամփոփում

Գլուխ 2-ում մանրամասն վերլուծվեց «ChessGameWebAssembly» նախագծի ճարտարապետական հենքը: Մենք տեսանք, թե ինչպես է համակարգը բաժանված շերտերի՝ ըստ Մաքուր ճարտարապետության սկզբունքների, և ինչպես են դիզայնի նախշերն օգնում կառավարել բարդ շախմատային տրամաբանությունը: Ներկայացված միկրոսերվիսային և Gateway մոդելները ցույց են տալիս նախագծի մասշտաբավորման հստակ ուղիները դեպի բարձր ծանրաբեռնվածությամբ համակարգեր:

ԳԼՈՒԽ 3 — ՀԻՄՆԱԿԱՆ ԼՈԳԻԿԱՅԻ ՆԱԽԱԳԾՈՒՄԸ ԵՎ ԻՐԱԿԱՆԱՑՈՒՄԸ

1.1. Դոմեյնի մոդելավորում և օբյեկտային-կողմնորոշված ճարտարապետություն

Ճախմատային խաղի ծրագրային իրականացումը պահանջում է խիստ կառուցվածքային մոտեցում, որտեղ իրական աշխարհի սուբյեկտները (խաղատախտակ, խաղաքարեր, կանոններ) պետք է վերածվեն վերացական մոդելների: Հիմնվելով **ChessGameWebAsseblly** նախագծի ուսումնասիրության վրա՝ կարելի է փաստել, որ համակարգի հիմքում ընկած է Դոմեյնի վրա հիմնված նախագծման (Domain-Driven Design - DDD) տարրեր պարունակող մոդելավորում:

Հիմնական տրամաբանության (Core Logic) մշակման ժամանակ առաջնային խնդիրն էր ստեղծել այնպիսի հիերարխիա, որը թույլ կտա ապահովել կոդի վերօգտագործելիությունը և հեշտ ընդլայնելիությունը: Ճախմատը, որպես համակարգ, բնութագրվում է բարձր կապակցվածությամբ (high coupling) իր տարրերի միջև, սակայն ծրագրային ճարտարապետության տեսանկյունից անհրաժեշտ է հասնել առավելագույն թույլ կապակցվածության (low coupling)՝ օգտագործելով ինկապսուլացիայի և պոլիմորֆիզմի սկզբունքները:

Նախագծի աղբյուրային կոդում կենտրոնական դեր են խաղում Models թղթապանակում զետեղված դասերը, որոնք սահմանում են խաղի էությունը: Հատկանշական է Position կառուցվածքի կամ դասի առկայությունը, որը հանդիսանում է ատոմար միավոր ամբողջ տրամաբանության համար:

1.2. Տարածական կոորդինատների և դիրքի մոդելավորում

Ճախմատային տախտակի վրա ցանկացած գործողություն սկսվում է կոորդինատների որոշումից: Նախագծում Position դասը հանդիսանում է այն հիմնաքարը, որի վրա կառուցվում են քայլերի հաշվարկման ալգորիթմները:

```
code C#
downloadcontent_copy
expand_less
public class Position
{
    public int Row { get; }
    public int Column { get; }

    public Position(int row, int column)
    {
        Row = row;
        Column = column;
    }

    public override bool Equals(object obj)
    {
        return obj is Position pos && pos.Row == Row && pos.Column == Column;
    }
}
```

```

public override int GetHashCode() => GetHashCode.Combine(Row, Column);

public static bool operator ==(Position left, Position right) =>
EqualityComparer<Position>.Default.Equals(left, right);
public static bool operator !=(Position left, Position right) => !(left == right);
}

```

Կողի վերլուծություն և ճարտարապետական հիմնավորում

Այս դասի իրականացումը ցույց է տալիս հեղինակի մոտեցումը տվյալների անփոփոխելիությանը (Immutability): Row և Column դաշտերը հանդիսանում են միայն ընթերցվող (readonly), ինչը չափազանց կարևոր է բազմահոսքանի միջավայրում կամ բարդ ալգորիթմական հաշվարկների ժամանակ սխալներից խուսափելու համար:

1. **Equals և GetHashCode.** Հավասարության ստուգման մեթոդների գերբեռնումը (overriding) թույլ է տալիս Position օբյեկտները օգտագործել որպես բանալիներ Dictionary կամ HashSet կառուցվածքներում, ինչն անհրաժեշտ է քայլերի վալիդացիայի և քեշավորման համար:
2. **Օպերատորների գերբեռնում.** == և != օպերատորների սահմանումը բարձրացնում է կողի ընթերցելիությունը՝ թույլ տալով համեմատել երկու դիրքեր պարզ սինտաքսով:

1.3. Խաղաքարերի հիերարխիան և պոլիմորֆիզմի կիրառումը

Նախագծի առանցքային ճարտարապետական լուծումներից մեկը վերացական Piece դասի ստեղծումն է: Սա թույլ է տալիս շախմատային շարժիչին աշխատել խաղաքարերի հետ՝ առանց իմանալու դրանց կոնկրետ տիպը (օրինակ՝ Թագուհի կամ Չինվոր), ինչը հանդիսանում է Օբյեկտային-կողմնորոշված նախագծման (OOP) դասական օրինակ:

```

public abstract class Piece
{
    public abstract PieceType Type { get; }
    public Player Color { get; }
    public bool HasMoved { get; set; } = false;

    protected Piece(Player color)
    {
        Color = color;
    }

    public abstract Piece Copy();
    public abstract IEnumerable<Position> GetPossibleMoves(Position from, Board board);
}

```

ճարտարապետական նկարագրություն

Piece դասը սահմանում է ընդհանուր վարքագիծ բոլոր խաղաքարերի համար.

- **PieceType.** Էնումերացիա (Enum), որը նույնականացնում է խաղաքարի տեսակը:
- **Color.** Որոշում է խաղաքարի պատկանելիությունը (Սպիտակ կամ Սև):
- **HasMoved.** Կարևոր դաշտ է այնպիսի քայլերի համար, ինչպիսիք են փոխադրումը (Castling) և գինվորի առաջին կրկնակի քայլը:
- **GetPossibleMoves.** Սա այն առանցքային վերացական մեթոդն է, որը յուրաքանչյուր ժամանակ դաս պետք է իրականացնի՝ հաշվի առնելով շախմատային կանոնները:

Այս մոդելը ապահովում է **Open/Closed Principle (SOLID)** սկզբունքը: Եթե մենք ցանկանանք ավելացնել նոր տիպի խաղաքար (օրինակ՝ շախմատային վարիացիաների համար), մեզ անհրաժեշտ չէ փոխել գոյություն ունեցող կոդը, այլ միայն ստեղծել նոր ժամանակ դաս:

1.4. Կոնկրետ խաղաքարերի տրամաբանության իրականացում

Դիտարկենք Knight (Ձի) խաղաքարի իրականացումը, որը շախմատում ունի ամենայուրահատուկ շարժման հետագիծը (L-աձև): Ի տարբերություն սահող խաղաքարերի (Թագուհի, Նավակ), Ձին կարող է «ցատկել» այլ խաղաքարերի վրայով:

```
public class Knight : Piece
{
    public override PieceType Type => PieceType.Knight;

    public Knight(Player color) : base(color) { }

    public override Piece Copy() => new Knight(Color) { HasMoved = HasMoved };

    public override IEnumerable<Position> GetPossibleMoves(Position from, Board board)
    {
        return MoveOffsets(from)
            .Where(pos => board.IsEmpty(pos) || board[pos].Color != Color);
    }

    private IEnumerable<Position> MoveOffsets(Position from)
    {
        int[] rowOffsets = { -2, -2, -1, -1, 1, 1, 2, 2 };
        int[] colOffsets = { -1, 1, -2, 2, -2, 2, -1, 1 };

        for (int i = 0; i < 8; i++)
        {
            yield return new Position(from.Row + rowOffsets[i], from.Column + colOffsets[i]);
        }
    }
}
```

Տեխնիկական վերլուծություն

Ձիու քայլերի գեներացման համար օգտագործվում է օֆսեթների (offsets) զանգվածների մեթոդը: Սա հաշվողական տեսանկյունից ամենաարդյունավետ ճանապարհն է:

1. **MoveOffsets.** Օգտագործում է yield return կոնստրուկցիան, ինչը թույլ է տալիս խնայել հիշողությունը՝ գեներացնելով կոորդինատները միայն ըստ անհրաժեշտության (Lazy evaluation):
2. **Linq-ի կիրառում.** Where ֆիլտրը միանգամից ստուգում է երկու պայման՝ արդյո՞ք թիրախային վանդակը դատարկ է, թե՞ այնտեղ գտնվում է հակառակորդի խաղաքարը: Սա դարձնում է կոդը կոմպակտ և ընթեռնելի:

1.5. Շախմատային տախտակի (Board) կառավարում և վիճակի պահպանում

Board դասը հանդիսանում է համակարգի «ուղեղը»: Այն պատասխանատու է խաղաքարերի դասավորվածության, դրանց տեղաշարժի և խաղի վիճակի փոփոխության համար:

```
code C#
downloadcontent_copy
expand_less
public class Board
{
    private readonly Piece[,] pieces = new Piece[8, 8];

    public Piece this[int r, int c]
    {
        get => pieces[r, c];
        set => pieces[r, c] = value;
    }

    public Piece this[Position pos]
    {
        get => this[pos.Row, pos.Column];
        set => this[pos.Row, pos.Column] = value;
    }

    public static Board Initial()
    {
        Board board = new Board();
        board.AddStartPieces();
        return board;
    }

    public bool IsEmpty(Position pos) => this[pos] == null;
```

```

public bool IsInBounds(Position pos)
{
    return pos.Row >= 0 && pos.Row < 8 && pos.Column >= 0 && pos.Column < 8;
}
}

```

Ինժեներական լուծումներ

1. **Ինդեքսատորներ (Indexers).** C#-ի ինդեքսատորների օգտագործումը (this[Position pos]) թույլ է տալիս աշխատել տախտակի հետ այնպես, ինչպես սովորական զանգվածի հետ, բայց օբյեկտային մակարդակում: Սա նվազեցնում է կոդի բարդությունը այլ մոդուլներում:
2. **Ներքին ներկայացում.** Օգտագործվում է երկչափ զանգված (Piece[,]): Չնայած Bitboard-երը (64-բիթանոց թվեր) ավելի արագ են շախմատային շարժիչների համար, օբյեկտային մոդելը ավելի նախընտրելի է Blazor WebAssembly-ի համար, քանի որ այն ուղղակիորեն քարտեզագրվում է UI բաղադրիչների հետ:

1.6. Քայլերի վալիդացիայի ալգորիթմ և «Շախի» ստուգում

Շախմատում ամենաբարդ խնդիրներից մեկը քայլի օրինականության ստուգումն է: Քայլը կարող է լինել տեսականորեն հնարավոր (ըստ խաղաքարի շարժման կանոնի), բայց գործնականում արգելված (եթե այն բացում է սեփական արքան շախի տակ):

```

code C#
downloadcontent_copy
expand_less
public bool IsLegalMove(Position from, Position to)
{
    Piece piece = this[from];
    if (piece == null) return false;

    // Սիմուլյացիա
    Board copy = this.Copy();
    copy.MakeMove(from, to);

    return !copy.IsInCheck(piece.Color);
}

```

Ալգորիթմական բարդություն

Այս մեթոդը օգտագործում է «Սիմուլյացիոն մոդելը»: Յուրաքանչյուր քայլ ստուգելու համար ստեղծվում է տախտակի ժամանակավոր պատճեն, կատարվում է քայլը, և ստուգվում է արքայի անվտանգությունը:

- **Առավելություններ.** Բացարձակ ճշգրտություն և կոդի պարզություն:
- **Մարտահրավերներ.** WebAssembly-ում հիշողության օպտիմիզացումը դառնում է առաջնային, քանի որ շատ պատճեններ ստեղծելը կարող է ազդել

արագագործության վրա: Նախագծում սա լուծված է Copy() մեթոդի արդյունավետ իրականացմամբ:

1.7. Սահող խաղաքարերի (Sliding Pieces) ընդհանուր տրամաբանություն

Թագուհին, Նավակը և Փիղը կիսում են նմանատիպ շարժման տրամաբանություն՝ նրանք շարժվում են ուղղություններով մինչև խոչընդոտի հանդիպելը: Նախագծում սա իրականացված է վերացական կամ օժանդակ մեթոդների միջոցով:

```
code C#
downloadcontent_copy
expand_less
protected IEnumerable<Position> GetMovesInDir(Position from, Board board, int rowDir, int colDir)
{
    for (int r = from.Row + rowDir, c = from.Column + colDir;
        board.IsInBounds(new Position(r, c));
        r += rowDir, c += colDir)
    {
        Position pos = new Position(r, c);
        if (board.IsEmpty(pos))
        {
            yield return pos;
            continue;
        }

        Piece piece = board[pos];
        if (piece.Color != Color)
        {
            yield return pos;
        }
        yield break;
    }
}
```

Այս մեթոդը հանդիսանում է «Clean Code»-ի օրինակ: Այն վերացարկում է ուղղությամբ շարժման տրամաբանությունը, ինչը թույլ է տալիս Rook (Նավակ) դասին ընդամենը կանչել այս մեթոդը 4 ուղղությունների համար, իսկ Bishop (Փիղ) դասին՝ անկյունագծերի համար:

1.8. Խաղի վիճակի կառավարում (Game State)

Խաղի ընթացքը կառավարելու համար օգտագործվում է GameState դասը, որը միավորում է տախտակը, ընթացիկ խաղացողին և խաղի կարգավիճակը (ընթացքի մեջ, հաղթանակ, ոչ-ոքի):

code C#

```

downloadcontent_copy
expand_less
public class GameState
{
    public Board Board { get; }
    public Player CurrentPlayer { get; private set; }
    public Result Result { get; private set; } = null;

    public GameState(Player player, Board board)
    {
        CurrentPlayer = player;
        Board = board;
    }

    public void MakeMove(Move move)
    {
        move.Execute(Board);
        CurrentPlayer = CurrentPlayer.Opponent();
        CheckGameOver();
    }
}

```

Դիզայնի Նմուշներ (Design Patterns)

Այստեղ նկատելի է **Command Pattern**-ի տարրեր: Քայլը (Move) ներկայացված է որպես օբյեկտ, որն ինքը գիտի, թե ինչպես փոփոխել տախտակը (move.Execute(Board)): Սա թույլ է տալիս հեշտությամբ իրականացնել քայլերի պատմությունը (History) և հետդարձը (Undo):

1.9. Չինվորի (Pawn) բարդ տրամաբանությունը

Չինվորը շախմատի ամենաբարդ կանոններ ունեցող խաղաքարն է (ուղիղ քայլ, անկյունագծով հարված, երկակի քայլ, en passant, փոխարկում): Նախագծի Pawn.cs ֆայլում այս ամենը մոդելավորված է առանձին պայմանական ստուգումների միջոցով, ինչը ապահովում է կանոնների 100% ճշգրտություն:

1.10. Եզրակացությունն գլխի վերաբերյալ

Այսպիսով, **ChessGameWebAssembly** նախագծի հիմնական տրամաբանությունը կառուցված է OOP սկզբունքների խիստ պահպանմամբ: C#-ի հզոր հնարավորությունները (Linq, Indexers, Immutability) թույլ են տվել ստեղծել շարժիչ, որը և՛ արագ է WebAssembly միջավայրում, և՛ հեշտ ընթերցման մշակողների համար: Հաջորդիվ կդիտարկվի, թե ինչպես է այս տրամաբանությունը ինտեգրվում Blazor-ի վիզուալ բաղադրիչների հետ:
